AKOSUA'S PHARMACY

Codebase Diagnosis & Proposed Design

SE01 Java Basics Lab

A review of the inherited application, the business risks identified, and a practical design for a safer and more maintainable rebuild.

Purpose of this document

I prepared this document to record my diagnosis of the original codebase before making implementation changes. I also included the class design and dispensing workflow I plan to follow for the rebuild.

1. Introduction

I inspected and ran the inherited application before making any changes. I wanted to understand how it behaves, identify defects that could affect pharmacy operations, and plan a structure that will be easier to maintain.

I found five top-level classes in PharmacyTracker.java: PharmacyTracker, Medication, Customer, Transaction and Pharmacy. The application supports over-the-counter and prescription medicines, walk-in and registered customers, dispensing, restocking and an end-of-day report.

2. Diagnosis findings

#	Finding	Code area	Root cause and reasoning	Business risk / impact	Severity
1	**Stock can become negative**	Pharmacy.dispense()	I found that the method assumes every requested quantity can be supplied. It does not first check that the pharmacy has enough stock.	The system can record a sale for medicine the pharmacy does not possess. Stock figures become unreliable and staff may promise customers unavailable medication.	**Critical**
2	**Business data is publicly mutable**	Medication; Customer; Transaction	I found that the classes do not control how their own data is changed. Other classes can directly alter important values without validation.	Stock, prices, customer details or transaction information could be changed accidentally or inconsistently. The pharmacy cannot confidently rely on its records.	**Critical**
3	**Input validation is missing**	Medication constructor; Pharmacy.dispense(); Pharmacy.restock()	I found that the application trusts callers to provide valid values. Validation is not performed where data enters the domain or an operation is requested.	A negative dispense quantity can increase stock; a negative restock quantity can reduce it. Invalid starting stock or prices can also corrupt inventory and financial information.	**High**
4	**Internal collections are exposed**	Pharmacy; Customer	I found that the application exposes its internal lists. Returning the original list also lets callers change records outside normal pharmacy operations.	Medication, customer history or transaction records could be removed or altered outside intended pharmacy operations. This weakens the reliability of the records.	**High**
5	**Categories are strings and rules sit in conditionals**	Pharmacy.dispense()	I found that categories and customer types are represented as strings. Category rules are concentrated in one method, where an unknown value can be handled incorrectly.	A spelling error or new medication category could be handled incorrectly. Adding controlled substances would make the central dispensing method more complex and risky to change.	**High**
6	**Pharmacy has too many responsibilities**	Pharmacy.dispense(); Pharmacy.printReport()	I found that business operations and presentation are combined. A pricing or reporting change could require edits to the same class that controls dispensing.	The application becomes harder to test, understand and extend. A small change can unintentionally affect unrelated pharmacy operations.	**Medium–High**

3. Runtime evidence

I ran the original program before making changes. The output below shows the stock problem I observed during that run:

```
Dispensed 2 x Paracetamol 500mg to Kofi Mensah for GHS 25.0
Dispensed 1 x Amoxicillin 250mg to Ama Owusu (prescribed by Dr. Boateng) for GHS 28.5
Restocked 50 x Paracetamol 500mg
Dispensed 100 x Amoxicillin 250mg to Kofi Mensah (prescribed by Dr. Boateng) for GHS 3000.0

=== Akosua's Pharmacy - end of day ===
Stock levels:
Paracetamol 500mg (OTC): 148 in stock
Amoxicillin 250mg (PRESCRIPTION): -61 in stock
Transactions today: 4
```

I traced the negative figure to the dispensing operation. Amoxicillin starts at 40 units; after one unit is dispensed, 39 remain. The next request is for 100 units, so the original code subtracts 100 and records ?61. I would reject that request before changing stock or records.

4. Most critical problem

I consider the most serious defect to be the ability to dispense more medicine than the pharmacy has available. This can make the records show a sale that the pharmacy could not supply. In my rebuild, I will check the requested quantity against available stock before changing stock, customer history or transaction records.

5. Proposed class design (UML)

The diagram shows the structure I plan to use for the rebuild. The numbered circles are explanatory callouts used in the description below; they are not standard UML symbols and do not indicate execution order.

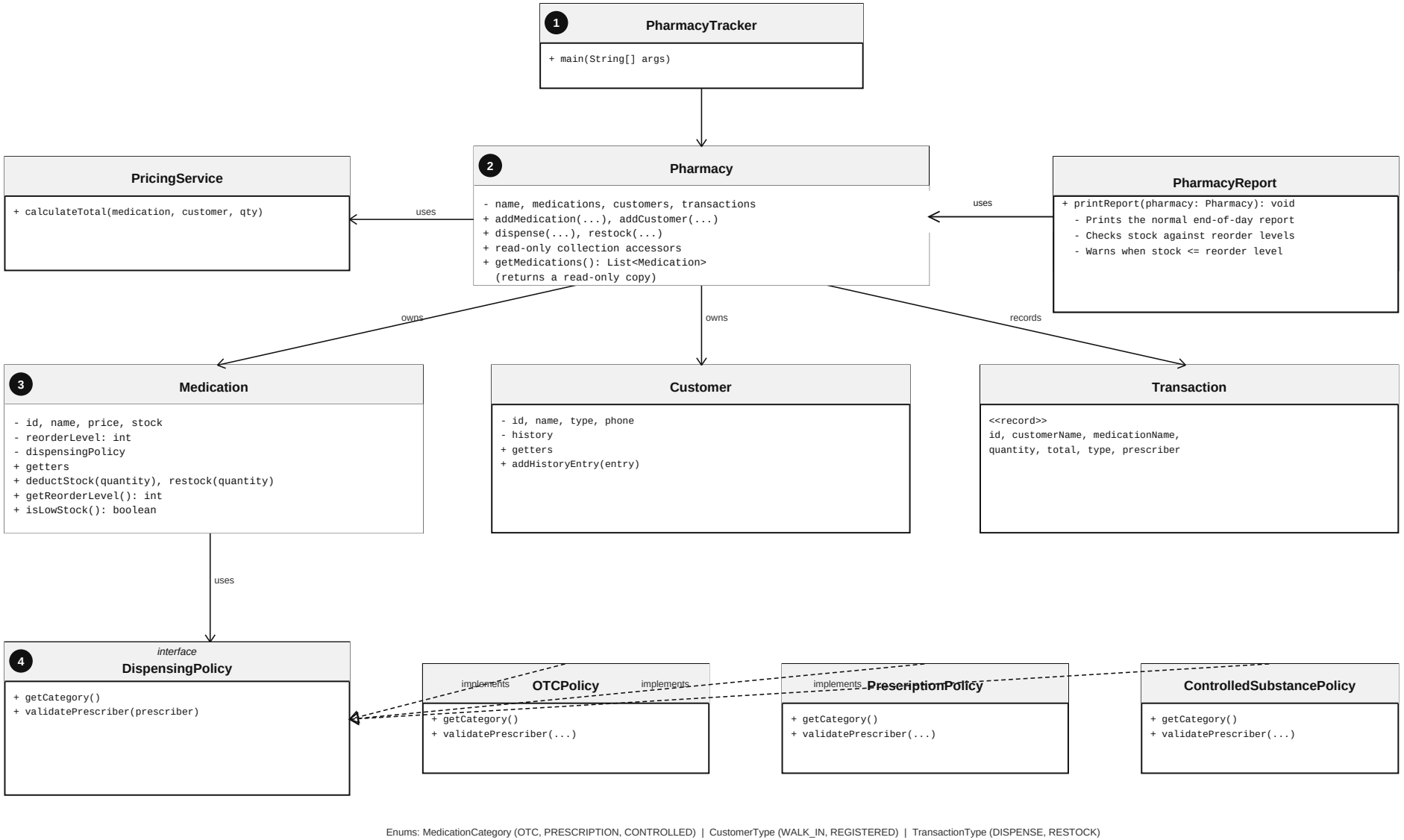

Figure 1. Proposed class structure for Akosua's Pharmacy.

5.1 Reading the diagram

No.	Part of the design	Purpose
1	**Application entry point · PharmacyTracker**	I will use it to start the program and create the objects needed for the demonstration.
2	**Pharmacy coordinator**	It will own the medication, customer and transaction collections. It will coordinate dispensing and restocking while providing read-only collection access.
3	**Domain objects**	Medication will protect stock and medication details; Customer will protect customer details and history; Transaction will store an immutable record of each operation.
4	**Policy abstraction**	DispensingPolicy will define a shared contract. OTC, prescription and controlled-substance policies will apply their own category rules. Detailed controlled-substance rules will follow the assignment or future requirements.
	Supporting collaborators	PricingService will calculate totals and the registered-customer discount. PharmacyReport will present stock and transaction information. Enums will represent medication, customer and transaction categories.

5.2 Why this structure is appropriate

I chose this structure so each part of the pharmacy system has a clear responsibility. Medication and Customer will protect their own data, while Pharmacy will coordinate the main operations. I will use DispensingPolicy for category-specific checks, keeping those rules out of one large dispensing method. PricingService and PharmacyReport will keep pricing and reporting separate from pharmacy operations, and enums will help avoid errors caused by spelling category and customer types as strings.

6. Dispensing workflow

I designed the workflow so that every check is completed before I change any business record. If a check fails, I will reject the request without changing stock, customer history or transactions.

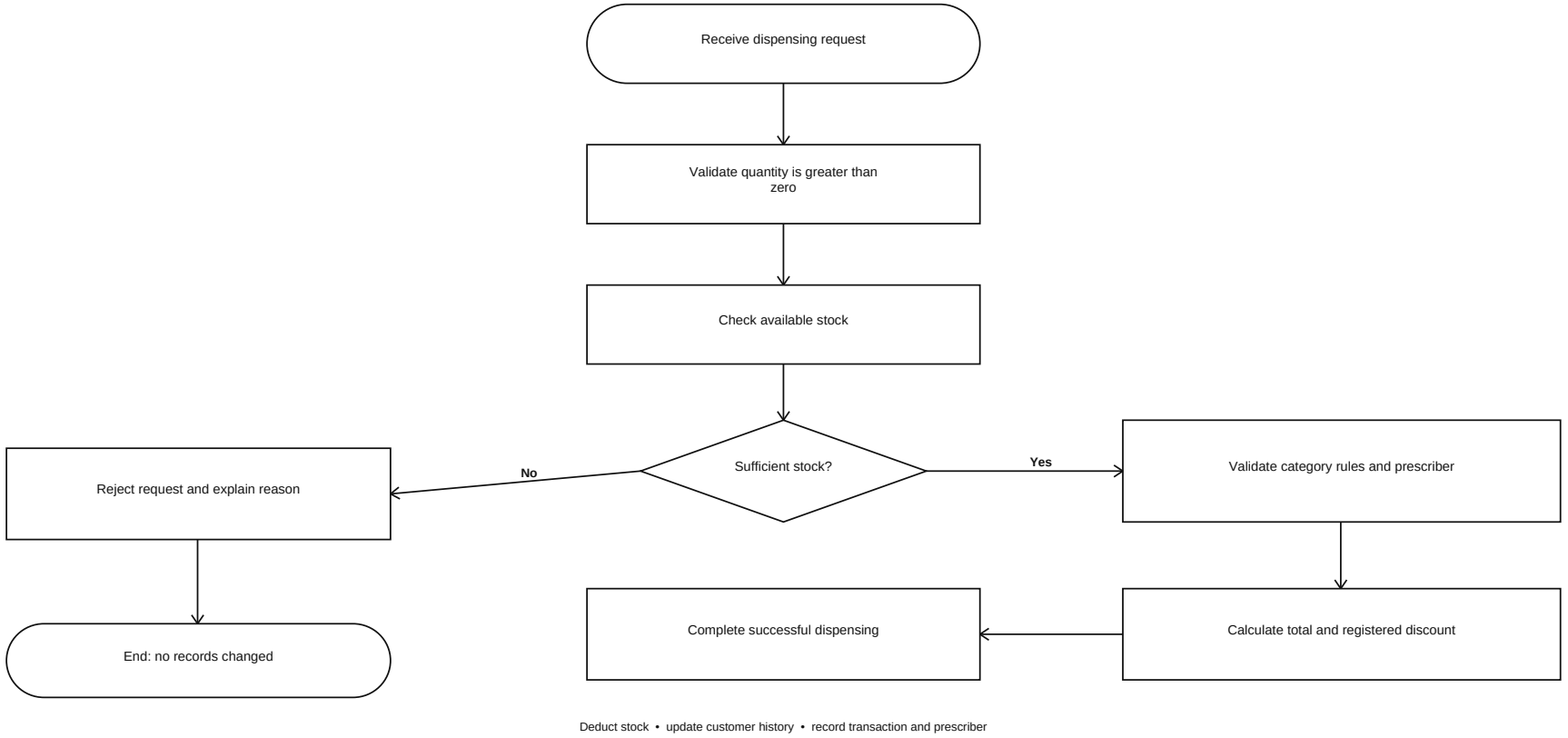

Figure 2. Proposed dispensing workflow. The left branch represents rejection; the right branch represents a successful operation.

Key: Rectangles represent actions, the diamond represents a decision, and rounded shapes represent the beginning or end of a path.

6.1 Workflow explanation

Step	Stage	What happens
1	**Receive request**	The pharmacy receives the customer, medication, requested quantity and prescriber details.
2	**Validate quantity**	Reject zero or negative quantities. A dispensing request must represent a positive number of units.
3	**Check stock**	Confirm that the requested quantity is no greater than the medication's available stock.
4	**Validate medication policy**	Apply the medication category's rules. Prescription and controlled-substance dispensing must require the relevant prescriber information.
5	**Calculate price**	Calculate the normal total and apply the existing 5% discount for registered customers.
6	**Complete operation**	After validation succeeds, deduct stock, update customer history and record the transaction. Retain the prescriber in the transaction for prescription and controlled-substance dispensing.
7	**Confirm or reject**	Display the appropriate outcome. A rejected request must leave stock, customer history and transaction records unchanged.

8. Additional improvement: low-stock alerts

I added a low-stock monitoring feature to the application. It checks medicine stock levels and identifies medicines that have fallen below the low-stock level. When a medicine is low in stock, the system alerts the pharmacist so they can restock it before it runs out.

Business value: This gives the pharmacist time to replenish medicines early, helps maintain availability for customers and reduces the risk of interruptions to the pharmacy's daily operations.